

BAB I PENDAHULUAN MOBILE PROGRAMMING	4
1.1 Pengertian Mobile Programming.....	4
BAB II SEJARAH MOBILE PROGRAMMING	5
2.1 Awal Perkembangan Perangkat Mobile.....	5
2.2 Era Symbian dan Java ME	6
2.3 Kemunculan iPhone dan Revolusi Mobile Modern	6
2.4 Kemunculan Android.....	7
2.5 Era Cross-Platform dan Framework Modern	7
BAB III PLATFORM MOBILE PROGRAMMING.....	8
3.1 Pengertian Platform Mobile.....	8
3.2 Android	8
3.3 iOS	9
3.4 Symbian.....	10
3.5 Windows Phone	11
3.6 BlackBerry OS.....	11
3.7 HarmonyOS	11
BAB IV ANALISIS PERBANDINGAN PLATFORM MOBILE	12
4.1 Perbandingan Android dan iOS.....	12
4.2 Perbandingan Android dan Symbian	12
4.3 Perbandingan iOS dan Windows Phone	12
BAB V FLUTTER DAN KOTLIN.....	12
5.1 Pengertian Flutter	12
5.2 Pengertian Kotlin.....	12
5.3 Perbedaan Flutter dan Kotlin.....	13
5.4 Kelebihan Flutter.....	13
5.5 Kekurangan Flutter	13
5.6 Kelebihan Kotlin	13
5.7 Kekurangan Kotlin	13
BAB VI ARSITEKTUR APLIKASI MOBILE	14
6.1 Konsep Arsitektur Mobile	14
6.2 MVC.....	14
6.3 MVVM	14
6.4 Clean Architecture	14

BAB VII TANTANGAN DAN MASA DEPAN MOBILE PROGRAMMING	14
7.1 Tantangan Mobile Programming	14
7.2 Masa Depan Mobile Programming	15
BAB IX ACTIVITY LIFECYCLE	15
9.1 Konsep Dasar Activity Lifecycle	15
9.2 Struktur Dasar Activity	16
9.3 Tahapan Lifecycle Activity	16
9.4 Lifecycle dan Manajemen Memori	18
9.5 Lifecycle dan Rotasi Layar	19
9.6 ViewModel dan Lifecycle	19
9.7 Kesimpulan Activity Lifecycle	19
BAB X GRADLE DAN PROJECT STRUCTURE	20
10.1 Pengertian Gradle	20
10.2 Fungsi Gradle	20
10.3 Struktur Project Android	20
10.4 Dependency Management.....	21
10.5 Build Variant.....	22
10.6 APK dan AAB	22
10.7 Gradle Kotlin DSL	22
10.8 Multi-Module Project.....	23
10.9 Signing Application.....	23
10.10 Kesimpulan Gradle dan Project.....	23
BAB XI LAYOUT DAN VIEW	23
11.1 Konsep Layout Android	23
11.2 LinearLayout.....	24
11.3 RelativeLayout.....	25
11.4 ConstraintLayout.....	25
11.5 FrameLayout	26
11.6 Pengertian View	26
11.7 TextView	27
11.8 EditText	27
11.9 Button	28
11.10 ImageView	29
11.11 RecyclerView.....	30
11.12 Material Design.....	31

BAB XII EVENT-DRIVEN PROGRAMMING	33
12.1 Pengertian Event-Driven Programming	33
12.2 Event pada Android.....	33
12.3 Event Listener	34
12.4 Click Event	34
12.5 Long Click Event	34
12.6 Touch Event	34
12.7 Callback Function	34
12.8 Event Queue.....	35
12.9 UI Thread.....	35
12.10 Asynchronous Programming.....	35
12.11 Event Handling pada RecyclerView.....	35
12.12 Gesture Detection	35
12.13 Kesimpulan Event-Driven.....	36
BAB XIII INTENT	36
13.1 Pengertian Intent	36
13.2 Jenis Intent	36
13.3 Explicit Intent	36
13.4 Implicit Intent.....	36
13.5 Mengirim Data Antar Activity	37
13.6 Intent Filter	37
13.7 Pending Intent.....	37
13.8 Intent dan Navigasi Modern	37
13.9 Intent dan Keamanan.....	37
13.10 Deep Link.....	38
13.11 Share Intent	38
13.12 Kesimpulan Intent.....	38
BAB VIV PENUTUP	39
DAFTAR PUSTAKA	40

BAB I PENDAHULUAN MOBILE PROGRAMMING

1.1 Pengertian Mobile Programming

Mobile programming merupakan suatu proses pengembangan perangkat lunak yang dirancang untuk berjalan pada perangkat bergerak seperti smartphone, tablet, wearable device, hingga perangkat embedded mobile lainnya. Dalam konteks perkembangan teknologi informasi modern, mobile programming menjadi salah satu cabang pengembangan perangkat lunak yang mengalami pertumbuhan paling pesat karena tingginya penggunaan perangkat mobile dalam kehidupan sehari-hari.

Perangkat mobile saat ini tidak hanya digunakan sebagai alat komunikasi, melainkan juga telah berkembang menjadi sarana untuk menjalankan berbagai aktivitas penting seperti transaksi keuangan, pembelajaran daring, hiburan digital, navigasi, kesehatan, hingga pengelolaan bisnis. Oleh karena itu, pengembangan aplikasi mobile menjadi bidang yang sangat strategis dalam dunia teknologi informasi.

Mobile programming berbeda dengan pemrograman desktop maupun web konvensional karena memiliki karakteristik khusus, seperti keterbatasan ukuran layar, keterbatasan daya baterai, keterbatasan sumber daya perangkat keras, serta kebutuhan konektivitas yang fleksibel. Selain itu, aplikasi mobile juga harus dirancang agar mampu memberikan pengalaman pengguna atau user experience yang baik pada berbagai ukuran perangkat.

Dalam mobile programming, seorang pengembang harus memahami berbagai aspek seperti desain antarmuka pengguna, pengelolaan memori perangkat, komunikasi jaringan, keamanan aplikasi, manajemen database lokal maupun cloud, serta integrasi dengan fitur perangkat keras seperti kamera, GPS, sensor, Bluetooth, NFC, dan fingerprint.

Perkembangan mobile programming juga melahirkan berbagai framework dan teknologi baru yang bertujuan mempermudah proses pengembangan aplikasi. Saat ini terdapat beberapa pendekatan utama dalam mobile development, yaitu native development, hybrid development, dan cross-platform development. Native development mengacu pada pengembangan aplikasi secara khusus untuk sistem operasi tertentu menggunakan bahasa pemrograman resmi platform tersebut. Hybrid development menggabungkan teknologi web dengan container mobile, sedangkan cross-platform development memungkinkan satu basis kode digunakan untuk berbagai sistem operasi sekaligus.

Kemunculan teknologi seperti Flutter, React Native, Kotlin Multiplatform, dan Xamarin menunjukkan bahwa dunia mobile programming terus mengalami inovasi yang sangat cepat. Hal tersebut menuntut

mahasiswa teknik informatika untuk memahami konsep dasar mobile programming secara mendalam agar mampu mengikuti perkembangan industri teknologi.

Selain aspek teknis, mobile programming juga memiliki pengaruh besar terhadap transformasi digital di berbagai sektor kehidupan. Banyak perusahaan, institusi pendidikan, rumah sakit, pemerintahan, dan organisasi bisnis kini lebih memprioritaskan pengembangan layanan berbasis mobile dibandingkan platform lainnya. Fenomena ini sering disebut sebagai mobile-first era.

Dalam dunia pendidikan tinggi, khususnya pada program studi Teknik Informatika, mata kuliah Mobile Programming memiliki tujuan untuk membekali mahasiswa dengan kemampuan merancang, membangun, menguji, dan memelihara aplikasi mobile yang efektif dan efisien. Mahasiswa diharapkan tidak hanya mampu membuat aplikasi sederhana, tetapi juga memahami arsitektur sistem mobile, siklus hidup aplikasi, keamanan data, optimasi performa, dan distribusi aplikasi.

BAB II SEJARAH MOBILE PROGRAMMING

2.1 Awal Perkembangan Perangkat Mobile

Perkembangan mobile programming tidak dapat dipisahkan dari sejarah perkembangan perangkat mobile itu sendiri. Pada awalnya, perangkat mobile hanya digunakan sebagai alat komunikasi suara. Telepon genggam generasi pertama atau 1G mulai diperkenalkan pada tahun 1980-an dengan kemampuan yang sangat terbatas. Teknologi ini masih menggunakan jaringan analog dan belum mendukung pengiriman data digital.

Pada masa tersebut, konsep aplikasi mobile belum berkembang sebagaimana saat ini. Perangkat telepon genggam hanya memiliki fungsi dasar seperti panggilan telepon dan penyimpanan kontak sederhana. Namun demikian, perkembangan teknologi jaringan seluler mulai membuka peluang bagi integrasi layanan digital pada perangkat bergerak.

Memasuki era 1990-an, muncul generasi kedua atau 2G yang menggunakan teknologi digital. Teknologi ini memungkinkan pengiriman pesan singkat atau SMS serta meningkatkan kualitas komunikasi. Pada periode ini mulai muncul perangkat Personal Digital Assistant atau PDA seperti Palm Pilot dan perangkat berbasis Windows CE.

PDA menjadi cikal bakal perangkat smartphone modern karena mulai menyediakan fungsi tambahan seperti kalender digital, catatan, email, dan sinkronisasi data dengan komputer. Pengembang perangkat lunak mulai menciptakan aplikasi sederhana untuk platform tersebut.

2.2 Era Symbian dan Java ME

Pada akhir tahun 1990-an hingga awal 2000-an, sistem operasi Symbian menjadi salah satu platform mobile paling populer di dunia. Symbian digunakan oleh berbagai produsen perangkat seperti Nokia, Sony Ericsson, dan Samsung. Sistem operasi ini dirancang khusus untuk perangkat mobile dengan keterbatasan memori dan daya baterai.

Pada masa kejayaan Symbian, pengembangan aplikasi mobile banyak dilakukan menggunakan bahasa C++ dan Java Micro Edition atau Java ME. Java ME menjadi teknologi yang sangat populer karena memungkinkan aplikasi berjalan pada berbagai jenis perangkat mobile dengan spesifikasi rendah.

Aplikasi Java ME biasanya memiliki ekstensi JAR dan dapat dijalankan pada feature phone. Walaupun kemampuan grafis dan performanya terbatas, teknologi ini menjadi fondasi awal pengembangan aplikasi mobile lintas perangkat.

Symbian memiliki kontribusi besar dalam sejarah mobile programming karena memperkenalkan konsep multitasking, pengelolaan aplikasi, serta distribusi aplikasi mobile melalui toko aplikasi sederhana. Namun, kompleksitas pengembangan aplikasi Symbian menjadi salah satu kelemahannya.

2.3 Kemunculan iPhone dan Revolusi Mobile Modern

Tahun 2007 menjadi titik penting dalam sejarah mobile programming ketika perusahaan `{ "company": "Apple Inc.", "location": "Cupertino, California, USA" }` memperkenalkan `{ "product": "iPhone", "category": "smartphone product line" }` pertama. Kehadiran iPhone mengubah paradigma penggunaan perangkat mobile secara drastis.

iPhone memperkenalkan antarmuka sentuh modern, pengalaman pengguna yang lebih intuitif, serta sistem operasi iOS yang stabil dan terintegrasi. Pada awalnya, Apple hanya memperbolehkan pengembangan aplikasi berbasis web. Namun kemudian Apple merilis Software Development Kit atau SDK resmi yang memungkinkan pengembang membuat aplikasi native menggunakan bahasa Objective-C.

Kemunculan App Store pada tahun 2008 menjadi revolusi besar dalam industri mobile programming. Pengembang dapat mendistribusikan aplikasi mereka secara global melalui satu platform terpusat. Model bisnis aplikasi mobile mulai berkembang pesat sejak saat itu.

2.4 Kemunculan Android

Setelah kesuksesan iPhone, perusahaan `{ "company": "Google", "Mountain View, California, USA" }` memperkenalkan sistem operasi Android pada tahun 2008. Android dikembangkan berdasarkan kernel Linux dan bersifat open source.

Android menawarkan fleksibilitas tinggi bagi produsen perangkat dan pengembang aplikasi. Bahasa pemrograman utama yang digunakan pada awal pengembangan Android adalah Java, kemudian berkembang menggunakan Kotlin.

Kehadiran Android menyebabkan pertumbuhan aplikasi mobile meningkat secara eksponensial karena banyak produsen smartphone mengadopsi sistem operasi tersebut. Android menjadi platform dengan pangsa pasar terbesar di dunia.

Android Studio kemudian diperkenalkan sebagai Integrated Development Environment atau IDE resmi Android. Platform ini menyediakan berbagai alat bantu pengembangan seperti emulator, debugger, profiler, dan sistem build otomatis.

2.5 Era Cross-Platform dan Framework Modern

Seiring meningkatnya kebutuhan pengembangan aplikasi untuk berbagai platform sekaligus, muncullah berbagai framework cross-platform seperti PhoneGap, Ionic, React Native, Xamarin, dan Flutter.

Framework cross-platform memungkinkan pengembang menulis satu basis kode untuk menghasilkan aplikasi Android maupun iOS secara bersamaan. Pendekatan ini dapat mengurangi waktu pengembangan dan biaya produksi.

Flutter yang dikembangkan oleh Google menjadi salah satu framework paling populer saat ini karena menawarkan performa tinggi dan antarmuka yang menarik. Flutter menggunakan bahasa pemrograman Dart dan memiliki rendering engine sendiri.

Di sisi lain, Kotlin berkembang menjadi bahasa resmi Android yang didukung penuh oleh Google. Kotlin menawarkan sintaks yang lebih modern, aman, dan ringkas dibandingkan Java.

Perkembangan teknologi mobile programming masih terus berlangsung hingga saat ini dengan munculnya teknologi seperti Progressive Web Apps, Artificial Intelligence pada perangkat mobile, Internet of Things, Augmented Reality, dan pengembangan aplikasi berbasis cloud.

BAB III PLATFORM MOBILE PROGRAMMING

3.1 Pengertian Platform Mobile

Platform mobile merupakan lingkungan perangkat lunak yang digunakan untuk menjalankan aplikasi pada perangkat bergerak. Platform ini terdiri atas sistem operasi, API, library, tools pengembangan, dan berbagai komponen pendukung lainnya.

Setiap platform mobile memiliki karakteristik, arsitektur, bahasa pemrograman, dan ekosistem yang berbeda. Pemahaman terhadap berbagai platform mobile sangat penting bagi pengembang aplikasi agar mampu menentukan teknologi yang tepat sesuai kebutuhan.

Beberapa platform mobile yang pernah maupun masih populer di dunia antara lain Android, iOS, Symbian, Windows Phone, BlackBerry OS, dan HarmonyOS.

3.2 Android

Sejarah Android

Android merupakan sistem operasi berbasis Linux yang dikembangkan oleh Google bersama Open Handset Alliance. Android pertama kali diperkenalkan pada tahun 2008 dan dengan cepat menjadi platform mobile paling dominan di dunia.

Android dirancang untuk memberikan fleksibilitas tinggi kepada produsen perangkat maupun pengembang aplikasi. Sistem operasi ini bersifat open source sehingga dapat dimodifikasi sesuai kebutuhan.

Karakteristik Android

Android memiliki berbagai karakteristik utama seperti open source, fleksibel, mendukung multitasking, memiliki integrasi layanan Google, serta mendukung berbagai jenis perangkat.

Android menyediakan Android SDK yang berisi library, emulator, API, dokumentasi, dan tools pengembangan lainnya. Pengembangan aplikasi Android saat ini banyak menggunakan bahasa Kotlin.

Kelebihan Android

Android memiliki banyak kelebihan dibandingkan platform lainnya. Salah satu keunggulan utama Android adalah sifatnya yang open source. Hal ini memungkinkan produsen perangkat maupun pengembang melakukan modifikasi sesuai kebutuhan.

Android juga memiliki jumlah pengguna terbesar di dunia sehingga memberikan peluang pasar yang sangat luas bagi pengembang aplikasi.

Selain itu, Android mendukung berbagai jenis perangkat dengan variasi harga yang beragam, mulai dari kelas entry-level hingga flagship.

Kelebihan lainnya adalah fleksibilitas distribusi aplikasi. Pengembang tidak hanya dapat mendistribusikan aplikasi melalui Google Play Store, tetapi juga melalui toko aplikasi pihak ketiga maupun instalasi langsung.

Android juga memiliki komunitas pengembang yang sangat besar sehingga mempermudah proses pembelajaran dan pengembangan aplikasi.

Kekurangan Android

Walaupun memiliki banyak kelebihan, Android juga memiliki beberapa kelemahan. Salah satunya adalah fragmentasi perangkat dan sistem operasi.

Banyaknya variasi perangkat menyebabkan pengembang harus memastikan aplikasi kompatibel dengan berbagai ukuran layar, spesifikasi hardware, dan versi Android.

Selain itu, karena sifatnya yang terbuka, Android memiliki risiko keamanan yang lebih tinggi dibandingkan iOS.

3.3 iOS

Sejarah iOS

iOS merupakan sistem operasi mobile yang dikembangkan oleh Apple untuk perangkat iPhone, iPad, dan iPod Touch.

iOS dikenal sebagai platform yang memiliki stabilitas tinggi dan pengalaman pengguna yang sangat baik.

Karakteristik iOS

iOS menggunakan ekosistem tertutup yang terintegrasi secara ketat antara perangkat keras dan perangkat lunak.

Bahasa pemrograman utama untuk pengembangan iOS adalah Swift dan Objective-C.

Apple menyediakan Xcode sebagai IDE resmi pengembangan aplikasi iOS.

Kelebihan iOS

Salah satu kelebihan utama iOS adalah stabilitas dan optimasi performanya. Karena Apple mengontrol perangkat keras dan perangkat lunak secara langsung, aplikasi dapat berjalan lebih optimal.

iOS juga dikenal memiliki keamanan yang tinggi. Proses review aplikasi di App Store dilakukan secara ketat sehingga mengurangi risiko malware.

Kelebihan lainnya adalah kualitas antarmuka pengguna yang konsisten.

Selain itu, pengguna iOS umumnya memiliki tingkat loyalitas dan daya beli yang tinggi sehingga menguntungkan bagi bisnis aplikasi premium.

Kekurangan iOS

Kekurangan utama iOS adalah sifatnya yang tertutup. Pengembang harus mengikuti aturan ketat dari Apple.

Biaya pengembangan aplikasi iOS juga relatif lebih tinggi karena memerlukan perangkat Mac untuk proses development.

Selain itu, distribusi aplikasi hanya dapat dilakukan melalui App Store kecuali menggunakan metode tertentu yang terbatas.

3.4 Symbian

Sejarah Symbian

Symbian merupakan salah satu sistem operasi mobile paling populer sebelum era Android dan iOS.

Symbian digunakan secara luas pada perangkat Nokia dan menjadi pemimpin pasar smartphone pada awal tahun 2000-an.

Karakteristik Symbian

Symbian dirancang untuk perangkat dengan sumber daya terbatas. Sistem operasi ini memiliki konsumsi daya yang efisien.

Pengembangan aplikasi Symbian umumnya menggunakan bahasa C++.

Kelebihan Symbian

Symbian memiliki efisiensi penggunaan memori dan baterai yang sangat baik.

Selain itu, Symbian mendukung multitasking pada masa ketika banyak platform lain belum memiliki kemampuan tersebut.

Kekurangan Symbian

Pengembangan aplikasi Symbian tergolong kompleks dan sulit dipelajari.

Antarmuka pengguna Symbian juga dianggap kurang modern dibandingkan Android dan iOS.

Keterlambatan inovasi menyebabkan Symbian kehilangan popularitas.

3.5 Windows Phone

Windows Phone dikembangkan oleh perusahaan `entity["company","Microsoft","Redmond, Washington, USA"]` sebagai penerus Windows Mobile.

Platform ini menawarkan antarmuka Metro UI yang unik dan ringan.

Bahasa pemrograman utama yang digunakan adalah C# dengan framework .NET.

Kelebihan Windows Phone adalah integrasi yang baik dengan layanan Microsoft seperti Office dan OneDrive.

Namun, platform ini gagal bersaing karena jumlah aplikasi yang sedikit dan rendahnya dukungan pengembang.

3.6 BlackBerry OS

BlackBerry OS pernah menjadi platform populer khususnya di kalangan bisnis.

Keunggulan utama BlackBerry terletak pada layanan email dan keamanan komunikasinya.

Namun, keterlambatan inovasi serta kurangnya aplikasi modern menyebabkan platform ini kehilangan pasar.

3.7 HarmonyOS

HarmonyOS merupakan sistem operasi yang dikembangkan oleh perusahaan `entity["company","Huawei","Shenzhen, Guangdong, China"]`.

Platform ini dikembangkan sebagai alternatif Android setelah pembatasan kerja sama dengan Google.

HarmonyOS dirancang untuk mendukung ekosistem Internet of Things dan berbagai perangkat pintar.

BAB IV ANALISIS PERBANDINGAN PLATFORM MOBILE

4.1 Perbandingan Android dan iOS

Android dan iOS merupakan dua platform mobile paling dominan di dunia saat ini. Kedua platform memiliki pendekatan yang berbeda dalam pengembangan maupun distribusi aplikasi. Android lebih terbuka dan fleksibel, sedangkan iOS lebih tertutup namun terintegrasi dengan baik. Dalam hal pangsa pasar global, Android unggul karena digunakan oleh banyak produsen perangkat. Namun dalam hal profit aplikasi, iOS sering kali lebih unggul karena pengguna iPhone cenderung lebih banyak melakukan pembelian aplikasi dan transaksi digital. Dari sisi keamanan, iOS memiliki sistem kontrol yang lebih ketat. Sedangkan Android unggul dalam fleksibilitas dan variasi perangkat.

4.2 Perbandingan Android dan Symbian

Android memiliki antarmuka yang lebih modern, dukungan aplikasi lebih luas, dan ekosistem yang terus berkembang. Sementara itu, Symbian unggul dalam efisiensi sumber daya pada perangkat dengan spesifikasi rendah. Namun, kompleksitas pengembangan dan kurangnya inovasi menyebabkan Symbian kalah bersaing.

4.3 Perbandingan iOS dan Windows Phone

iOS memiliki ekosistem aplikasi yang jauh lebih besar dibandingkan Windows Phone. Windows Phone sebenarnya memiliki performa yang ringan dan antarmuka yang menarik. Namun, kurangnya dukungan aplikasi menjadi faktor utama kegagalannya.

BAB V FLUTTER DAN KOTLIN

5.1 Pengertian Flutter

Flutter merupakan framework open source yang dikembangkan oleh Google untuk membangun aplikasi cross-platform. Flutter memungkinkan pengembang membuat aplikasi Android, iOS, web, desktop, dan embedded menggunakan satu basis kode. Flutter menggunakan bahasa pemrograman Dart. Framework ini memiliki rendering engine sendiri sehingga tampilan antarmuka dapat konsisten di berbagai platform. Flutter juga mendukung fitur hot reload yang memungkinkan perubahan kode langsung terlihat tanpa melakukan build ulang secara penuh.

5.2 Pengertian Kotlin

Kotlin merupakan bahasa pemrograman modern yang dikembangkan oleh JetBrains. Google secara resmi menjadikan Kotlin sebagai bahasa utama pengembangan Android. Kotlin dirancang agar

kompatibel dengan Java namun memiliki sintaks yang lebih ringkas dan aman.Kotlin mendukung paradigma pemrograman object-oriented dan functional programming.

5.3 Perbedaan Flutter dan Kotlin

Walaupun sering dibandingkan, Flutter dan Kotlin sebenarnya memiliki fungsi yang berbeda.Flutter merupakan framework pengembangan aplikasi, sedangkan Kotlin adalah bahasa pemrograman.Flutter digunakan untuk membangun aplikasi cross-platform menggunakan Dart.Sementara itu, Kotlin umumnya digunakan untuk pengembangan native Android.Dari sisi performa, aplikasi Kotlin native biasanya memiliki integrasi lebih mendalam dengan sistem Android.Namun Flutter menawarkan efisiensi pengembangan karena satu kode dapat digunakan pada berbagai platform.Flutter juga unggul dalam pengembangan antarmuka yang konsisten dan modern.Sedangkan Kotlin memiliki akses lebih lengkap terhadap fitur native Android.Dalam hal learning curve, Flutter memerlukan pemahaman terhadap Dart dan konsep widget.Sementara Kotlin lebih mudah dipelajari bagi pengembang yang sudah memahami Java.

5.4 Kelebihan Flutter

Flutter memiliki banyak kelebihan seperti pengembangan cepat, tampilan antarmuka menarik, performa tinggi, dan dukungan cross-platform.Flutter juga memiliki komunitas yang berkembang pesat.

Selain itu, Flutter mendukung hot reload sehingga mempercepat proses debugging.

5.5 Kekurangan Flutter

Ukuran aplikasi Flutter cenderung lebih besar dibandingkan aplikasi native.Selain itu, beberapa fitur native tertentu mungkin memerlukan integrasi tambahan.

5.6 Kelebihan Kotlin

Kotlin memiliki sintaks yang modern, aman dari null pointer exception, dan kompatibel penuh dengan Java.Kotlin juga memberikan performa native yang optimal pada Android.

5.7 Kekurangan Kotlin

Kotlin lebih fokus pada pengembangan Android sehingga tidak sefleksibel Flutter untuk cross-platform.

Selain itu, pengembangan multi-platform dengan Kotlin masih belum sepopuler Flutter.

BAB VI ARSITEKTUR APLIKASI MOBILE

6.1 Konsep Arsitektur Mobile

Arsitektur aplikasi mobile merupakan struktur atau pola pengorganisasian komponen aplikasi agar sistem mudah dikembangkan, dipelihara, dan diuji. Arsitektur yang baik sangat penting untuk menjaga kualitas aplikasi terutama ketika aplikasi semakin kompleks. Beberapa arsitektur populer dalam mobile programming antara lain MVC, MVP, MVVM, dan Clean Architecture.

6.2 MVC

Model View Controller atau MVC merupakan pola arsitektur yang membagi aplikasi menjadi tiga bagian utama yaitu model, view, dan controller. Model bertugas mengelola data dan logika bisnis. View bertugas menampilkan antarmuka pengguna. Controller bertugas menghubungkan model dan view.

6.3 MVVM

Model View ViewModel atau MVVM banyak digunakan pada Android modern. Arsitektur ini memisahkan logika bisnis dari tampilan sehingga kode menjadi lebih terstruktur. MVVM biasanya dikombinasikan dengan LiveData dan ViewModel.

6.4 Clean Architecture

Clean Architecture bertujuan menciptakan aplikasi yang modular dan mudah diuji. Arsitektur ini memisahkan aplikasi menjadi beberapa layer seperti presentation layer, domain layer, dan data layer.

BAB VII TANTANGAN DAN MASA DEPAN MOBILE PROGRAMMING

7.1 Tantangan Mobile Programming

Pengembangan aplikasi mobile menghadapi berbagai tantangan seperti fragmentasi perangkat, keamanan data, optimasi baterai, kompatibilitas sistem operasi, dan perubahan teknologi yang sangat cepat. Pengembang juga harus memperhatikan pengalaman pengguna agar aplikasi tetap mudah digunakan. Selain itu, keamanan menjadi isu yang sangat penting karena aplikasi mobile sering mengelola data sensitif pengguna.

7.2 Masa Depan Mobile Programming

Mobile programming diperkirakan akan terus berkembang seiring kemajuan teknologi seperti Artificial Intelligence, Machine Learning, Internet of Things, Blockchain, dan Augmented Reality. Aplikasi mobile masa depan diprediksi akan semakin cerdas, terintegrasi, dan personal. Teknologi jaringan 5G juga akan meningkatkan kemampuan aplikasi mobile dalam menangani data real-time. Selain itu, perkembangan wearable device dan smart device akan memperluas ruang lingkup mobile programming.

BAB IX ACTIVITY LIFECYCLE

9.1 Konsep Dasar Activity Lifecycle

Dalam pengembangan aplikasi Android menggunakan Kotlin, Activity merupakan salah satu komponen utama yang memiliki peran sebagai penghubung antara aplikasi dan pengguna. Setiap layar yang muncul pada aplikasi Android umumnya direpresentasikan sebagai sebuah Activity. Karena aplikasi Android berjalan pada perangkat dengan sumber daya terbatas seperti memori, CPU, dan baterai, sistem Android memerlukan mekanisme khusus untuk mengatur bagaimana Activity dibuat, dijalankan, dihentikan sementara, hingga dihancurkan.

Mekanisme tersebut dikenal sebagai Activity Lifecycle. Lifecycle atau siklus hidup Activity merupakan urutan status yang dialami sebuah Activity sejak pertama kali dibuat hingga dihancurkan oleh sistem operasi Android.

Lifecycle menjadi salah satu konsep paling penting dalam Android karena hampir seluruh aplikasi modern bergantung pada pengelolaan lifecycle yang baik. Kesalahan dalam memahami lifecycle dapat menyebabkan berbagai masalah seperti memory leak, aplikasi crash, kehilangan data, performa lambat, dan konsumsi baterai berlebihan.

Android menggunakan lifecycle untuk memastikan penggunaan resource perangkat berlangsung secara efisien. Ketika pengguna membuka aplikasi lain, menerima panggilan telepon, atau mengunci layar perangkat, Activity akan mengalami perubahan status. Android akan memanggil method tertentu sesuai kondisi Activity.

Dengan memahami lifecycle secara mendalam, pengembang dapat membuat aplikasi yang lebih stabil, responsif, hemat resource, dan nyaman digunakan.

9.2 Struktur Dasar Activity

Dalam Kotlin, sebuah Activity biasanya dibuat dengan mewarisi class AppCompatActivity.

Contoh sederhana:

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

Kode tersebut menunjukkan bahwa MainActivity menggunakan lifecycle Android. Ketika Activity dijalankan, Android akan memanggil berbagai callback method sesuai kondisi aplikasi.

9.3 Tahapan Lifecycle Activity

onCreate()

Method onCreate() merupakan method pertama yang dipanggil ketika Activity dibuat.

Tahap ini digunakan untuk melakukan inisialisasi awal aplikasi.

Beberapa proses yang umum dilakukan pada onCreate() antara lain:

- Menghubungkan layout XML
- Inisialisasi variabel
- Binding View
- Setup RecyclerView
- Setup database
- Inisialisasi ViewModel
- Setup event listener

Contoh:

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
}
```

Method ini hanya dipanggil sekali selama Activity dibuat.

onStart()

Method onStart() dipanggil ketika Activity mulai terlihat oleh pengguna.

Pada tahap ini Activity sudah muncul di layar namun belum aktif sepenuhnya.

Biasanya digunakan untuk:

- Registrasi listener
- Persiapan data
- Memulai koneksi ringan

onResume()

Method onResume() dipanggil ketika Activity aktif digunakan pengguna.

Pada kondisi ini Activity berada di foreground.

Event input pengguna seperti sentuhan layar dan keyboard mulai diterima.

Biasanya digunakan untuk:

- Menjalankan animasi
- Mengaktifkan sensor
- Mengaktifkan kamera
- Memutar audio/video

onPause()

Method onPause() dipanggil ketika Activity kehilangan fokus sementara.

Contohnya ketika pengguna membuka dialog atau notifikasi.

Pada tahap ini pengembang biasanya:

- Menyimpan data sementara
- Menghentikan animasi
- Pause media player
- Menyimpan state aplikasi

onStop()

Method onStop() dipanggil ketika Activity tidak lagi terlihat.

Biasanya digunakan untuk:

- Menutup koneksi database
- Menghentikan proses berat
- Melepaskan resource

onRestart()

Method ini dipanggil ketika Activity kembali dijalankan setelah berhenti.

onDestroy()

Method terakhir sebelum Activity dihancurkan.

Digunakan untuk:

- Membersihkan memori
- Menutup koneksi
- Menghapus listener
- Menghentikan thread

9.4 Lifecycle dan Manajemen Memori

Android memiliki sistem manajemen memori otomatis.

Ketika memori perangkat penuh, Android dapat menghentikan aplikasi di background.

Karena itu pengembang harus memahami lifecycle agar aplikasi tetap aman.

Kesalahan umum pada lifecycle antara lain:

- Tidak menutup koneksi database
- Tidak menghentikan thread
- Menyimpan context terlalu lama
- Menjalankan proses berat pada UI thread

Kesalahan tersebut dapat menyebabkan memory leak.

9.5 Lifecycle dan Rotasi Layar

Rotasi layar merupakan salah satu tantangan dalam Android.

Ketika perangkat diputar, Activity biasanya akan dihancurkan lalu dibuat ulang.

Akibatnya data dapat hilang jika tidak dikelola dengan baik.

Untuk mengatasi masalah ini digunakan:

- ViewModel
- onSaveInstanceState()
- LiveData
- SavedStateHandle

9.6 ViewModel dan Lifecycle

Android Jetpack menyediakan ViewModel untuk mempertahankan data ketika Activity direcreate.

ViewModel menjadi solusi modern untuk pengelolaan lifecycle.

Selain itu terdapat LifecycleOwner dan LifecycleObserver.

Komponen tersebut membantu pengembang membuat aplikasi lebih modular.

9.7 Kesimpulan Activity Lifecycle

Activity Lifecycle merupakan fondasi utama pengembangan Android.

Pemahaman lifecycle sangat penting untuk menciptakan aplikasi yang stabil, aman, hemat resource, dan optimal.

Pengembang Android modern wajib memahami hubungan lifecycle dengan UI, database, networking, sensor, dan asynchronous programming.

BAB X GRADLE DAN PROJECT STRUCTURE

10.1 Pengertian Gradle

Gradle merupakan build automation tool yang digunakan Android Studio untuk mengelola proses build aplikasi Android. Dalam project Android modern, Gradle memiliki peran sangat penting karena seluruh proses compile, dependency management, signing, hingga packaging aplikasi dilakukan menggunakan Gradle. Gradle dikembangkan untuk mempermudah pengelolaan project skala besar. Sebelum Gradle populer, banyak project menggunakan Ant dan Maven. Namun Gradle lebih fleksibel dan mendukung scripting modern. Android Studio secara otomatis menghasilkan konfigurasi Gradle ketika membuat project baru.

10.2 Fungsi Gradle

Gradle memiliki berbagai fungsi utama seperti:

- Mengelola dependency library
- Mengatur versi SDK
- Build APK dan AAB
- Menjalankan testing
- Signing aplikasi
- Minify code
- Multi-module project
- Konfigurasi build variant

Tanpa Gradle, pengembang harus mengelola seluruh library secara manual.

10.3 Struktur Project Android

Project Android memiliki struktur folder tertentu.

Folder java atau kotlin

Berisi seluruh source code aplikasi.

Folder res

Folder resource berisi:

- layout
- drawable
- mipmap
- values
- menu
- anim

AndroidManifest.xml

Berfungsi mendeklarasikan komponen aplikasi.

build.gradle

Berisi konfigurasi build aplikasi.

Gradle Scripts

Digunakan untuk konfigurasi sistem build.

10.4 Dependency Management

Dependency merupakan library tambahan yang digunakan aplikasi.

Contoh dependency:

```
implementation 'androidx.recyclerview:recyclerview:1.3.0'
```

Dependency mempermudah pengembang menggunakan fitur modern.

Contoh library populer:

- Retrofit
- Glide
- Room

- Firebase
- Coroutines
- Navigation Component

10.5 Build Variant

Android mendukung beberapa build variant.

Debug

Digunakan saat pengembangan.

Release

Digunakan untuk distribusi ke pengguna.

Build release biasanya menggunakan:

- Obfuscation
- Minify
- Proguard
- Signing Key

10.6 APK dan AAB

APK merupakan format instalasi Android tradisional.

AAB atau Android App Bundle merupakan format distribusi modern.

Google Play kini lebih memprioritaskan AAB.

AAB menghasilkan ukuran aplikasi lebih kecil.

10.7 Gradle Kotlin DSL

Android modern mulai menggunakan Kotlin DSL.

Contoh:

```
plugins {  
    id("com.android.application")  
}
```

Kotlin DSL lebih aman dan mendukung autocomplete.

10.8 Multi-Module Project

Project besar biasanya dibagi menjadi beberapa module.

Keuntungan multi-module:

- Kode lebih modular
- Build lebih cepat
- Maintenance lebih mudah

10.9 Signing Application

Aplikasi Android harus ditandatangani menggunakan signing key. Signing digunakan untuk keamanan dan identitas aplikasi. Tanpa signing, aplikasi tidak dapat dipublikasikan.

10.10 Kesimpulan Gradle dan Project

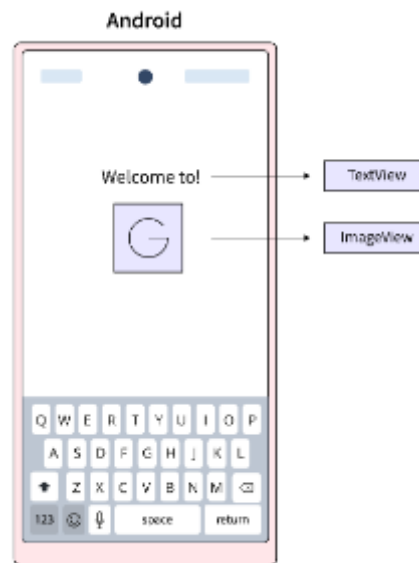
Gradle merupakan fondasi build system Android modern.

Pemahaman terhadap struktur project dan Gradle sangat penting agar pengembang mampu membangun aplikasi secara profesional.

BAB XI LAYOUT DAN VIEW

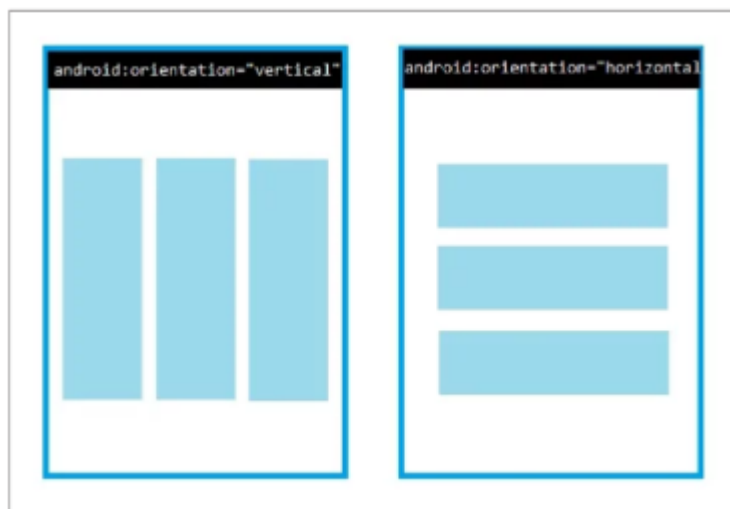
11.1 Konsep Layout Android

Layout merupakan struktur tampilan aplikasi Android. Layout menentukan posisi dan hubungan antar komponen antarmuka. Android menggunakan XML untuk mendefinisikan layout. Layout menjadi bagian penting karena berkaitan langsung dengan user experience. Desain layout yang buruk dapat menyebabkan aplikasi sulit digunakan.



11.2 LinearLayout

LinearLayout merupakan layout paling sederhana. Komponen disusun secara horizontal atau vertikal.



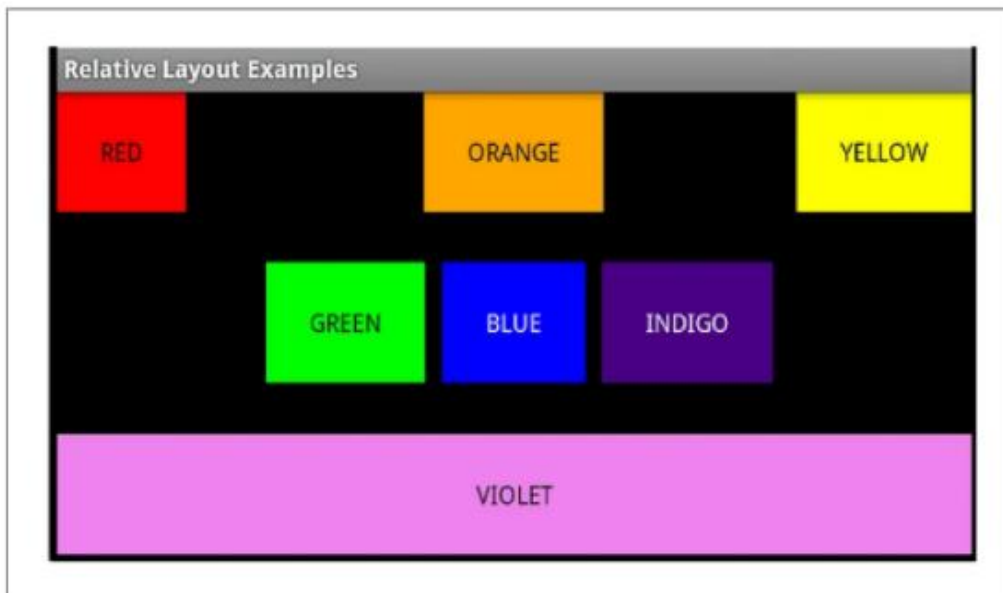
Contoh:

```
<LinearLayout  
    android:orientation="vertical">  
</LinearLayout>
```

LinearLayout mudah dipahami namun kurang optimal untuk desain kompleks.

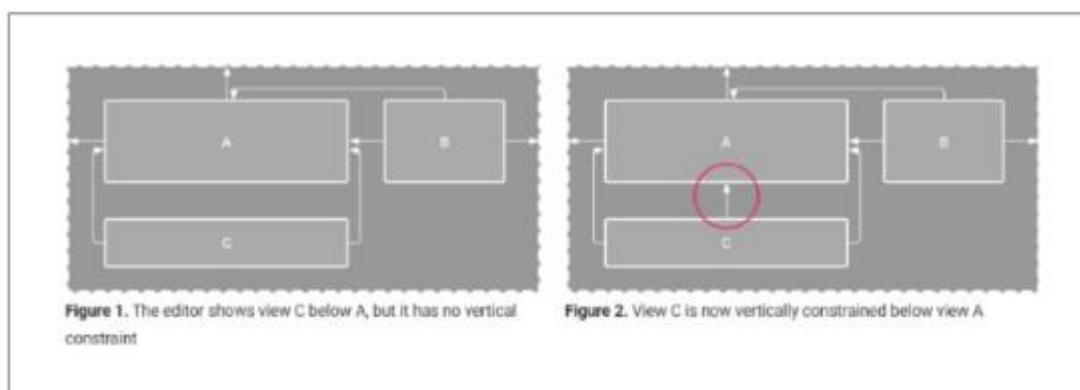
11.3 RelativeLayout

RelativeLayout memungkinkan posisi komponen berdasarkan komponen lain. Layout ini populer sebelum ConstraintLayout. Namun penggunaan RelativeLayout berlebihan dapat menurunkan performa.



11.4 ConstraintLayout

ConstraintLayout merupakan layout modern Android. ConstraintLayout sangat fleksibel dan efisien.



Keuntungan ConstraintLayout:

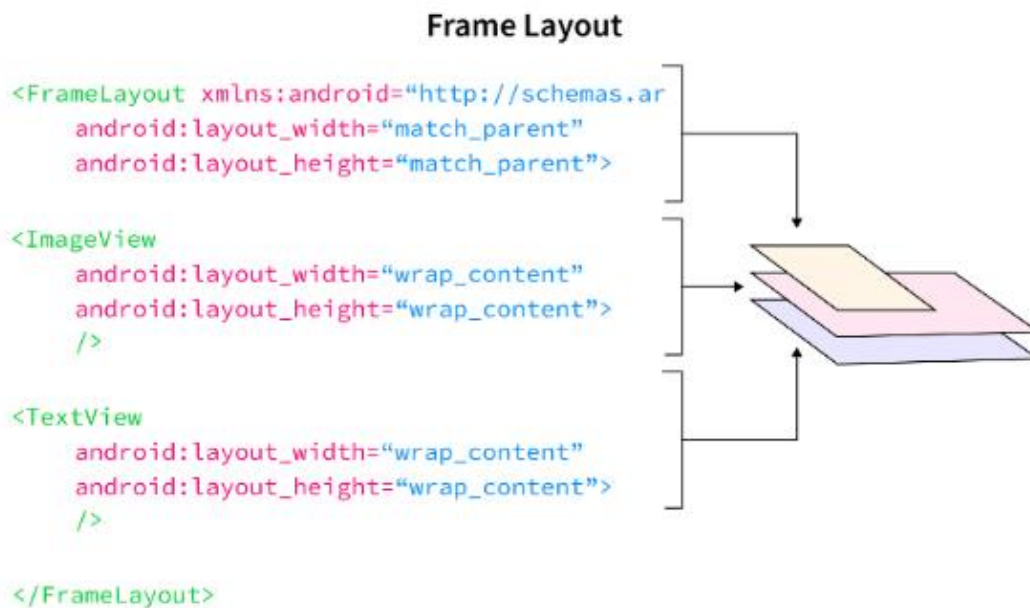
- Flat hierarchy
- Performa baik
- Mendukung desain kompleks

- Mudah membuat responsive UI

ConstraintLayout menjadi standar modern Android.

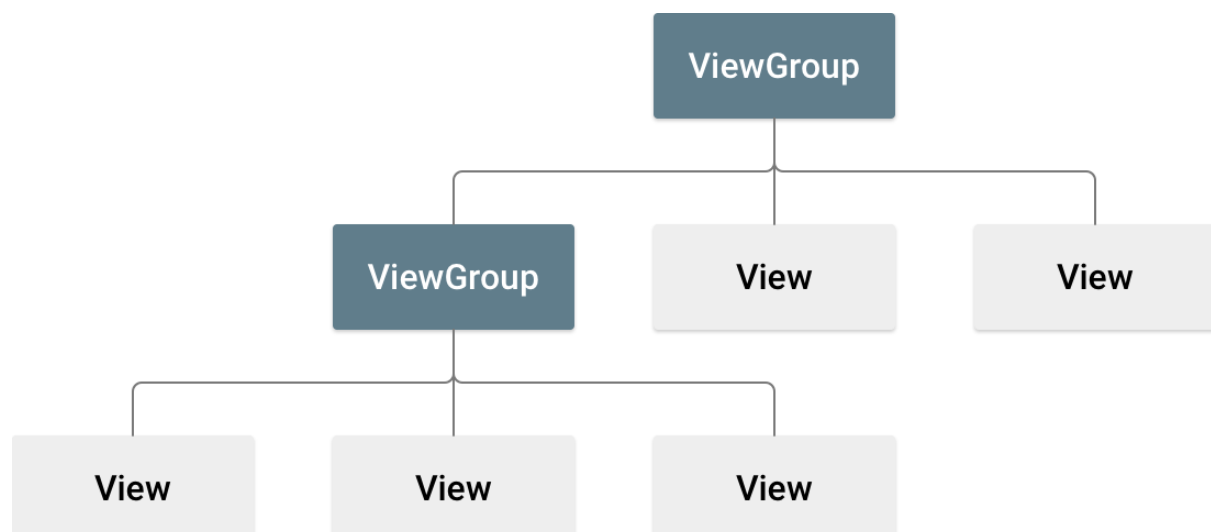
11.5 FrameLayout

FrameLayout digunakan untuk menumpuk komponen. Umumnya digunakan pada Fragment.



11.6 Pengertian View

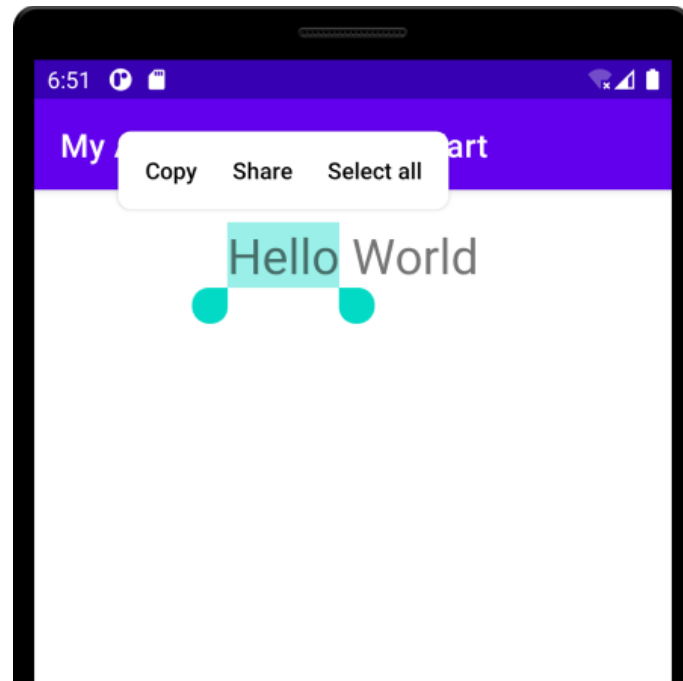
View merupakan komponen dasar antarmuka Android.



Semua elemen UI merupakan turunan dari class View.

11.7 TextView

TextView digunakan menampilkan teks.

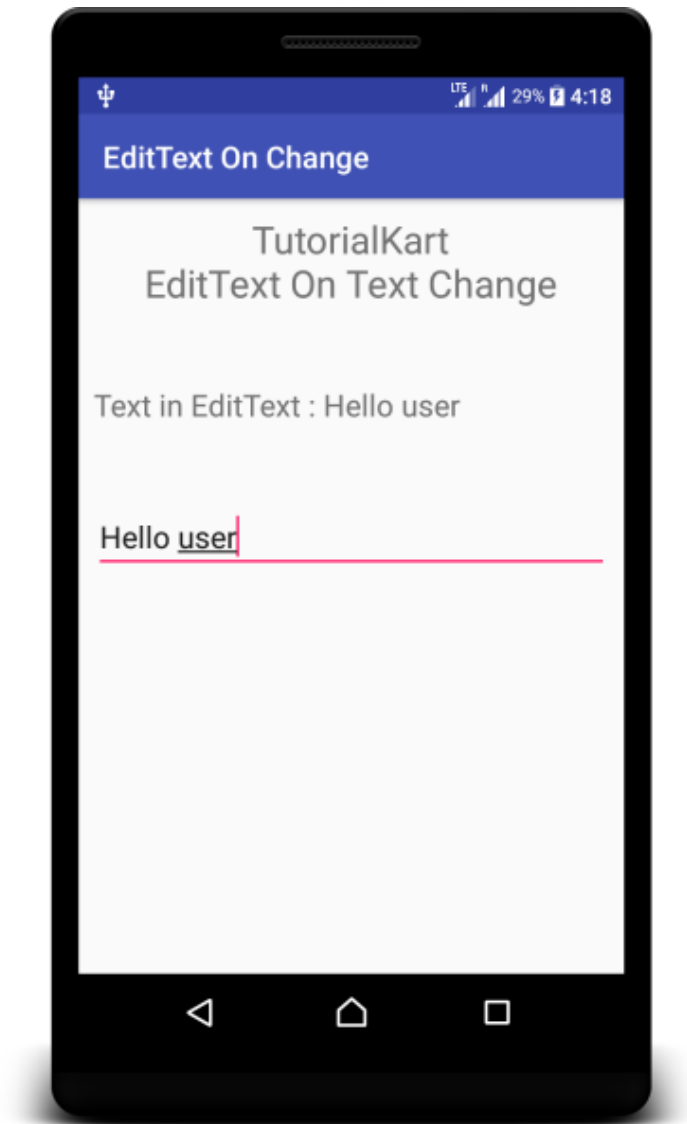


TextView mendukung:

- Font
- Warna
- Style
- Alignment
- Spannable Text

11.8 EditText

EditText digunakan menerima input pengguna.



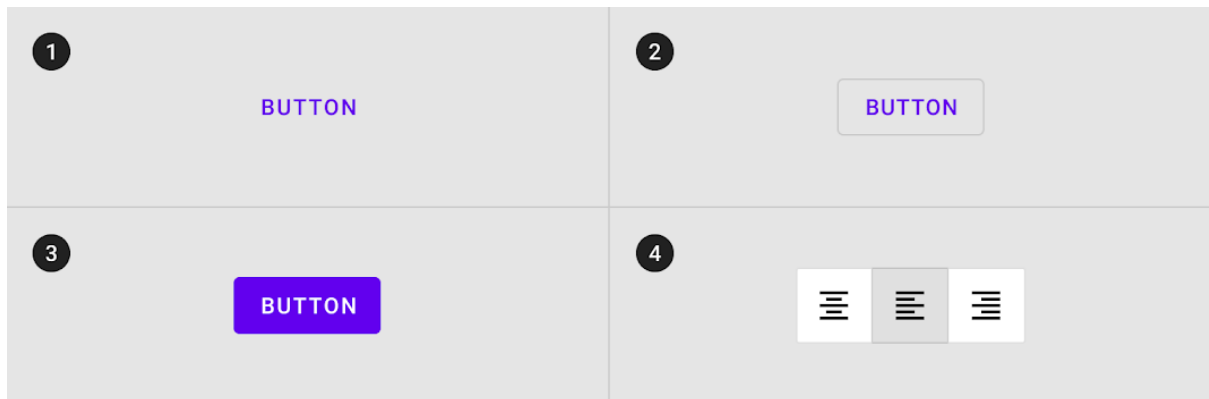
Komponen ini penting pada:

- Login
- Registrasi
- Pencarian
- Form data

11.9 Button

Button digunakan untuk menjalankan aksi.

Button biasanya dikombinasikan dengan event listener.



11.10 ImageView

ImageView digunakan menampilkan gambar.

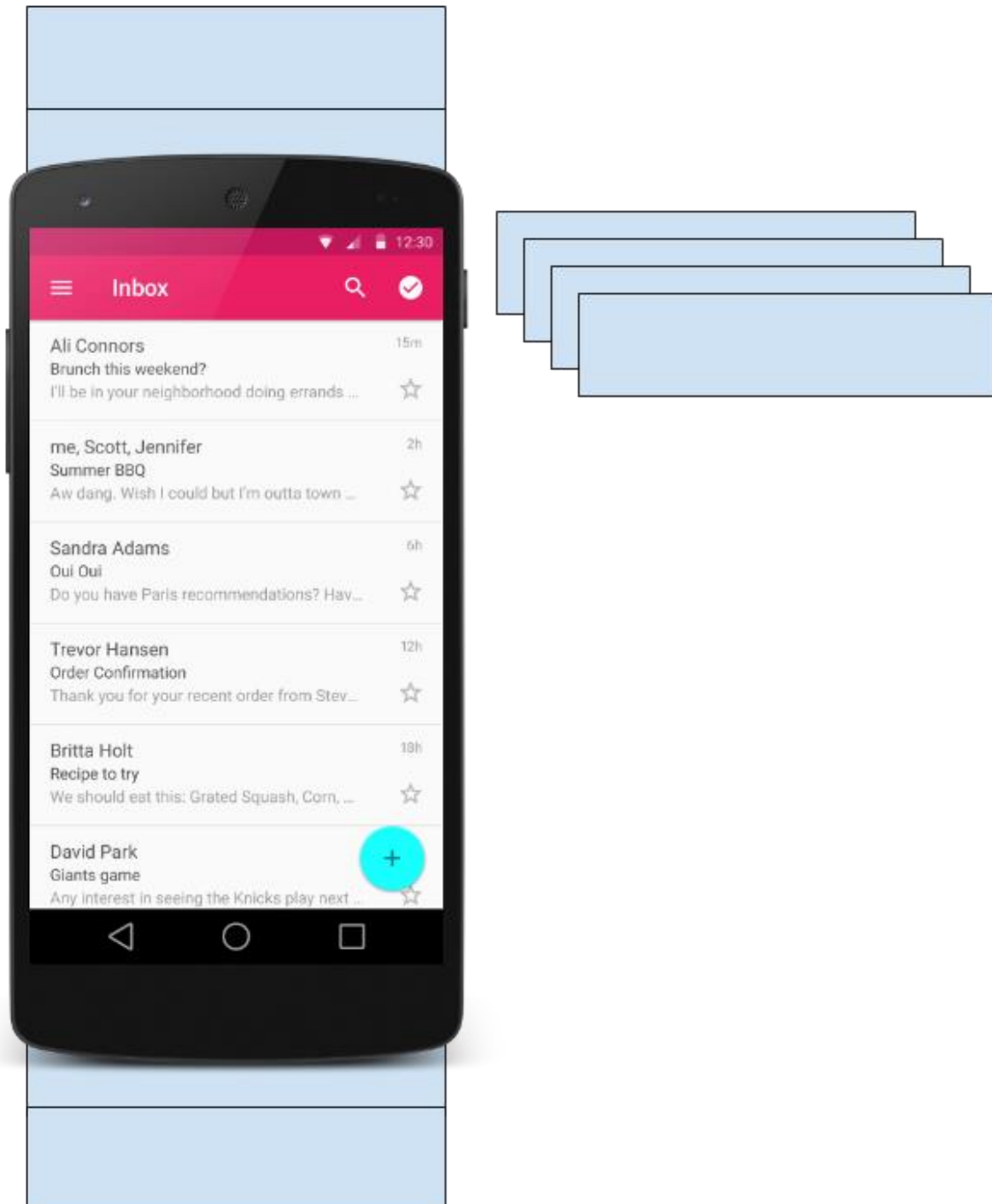
Android mendukung:

- PNG
- JPG
- WEBP
- SVG tertentu



11.11 RecyclerView

RecyclerView digunakan untuk menampilkan daftar data.



RecyclerView lebih efisien dibanding ListView.

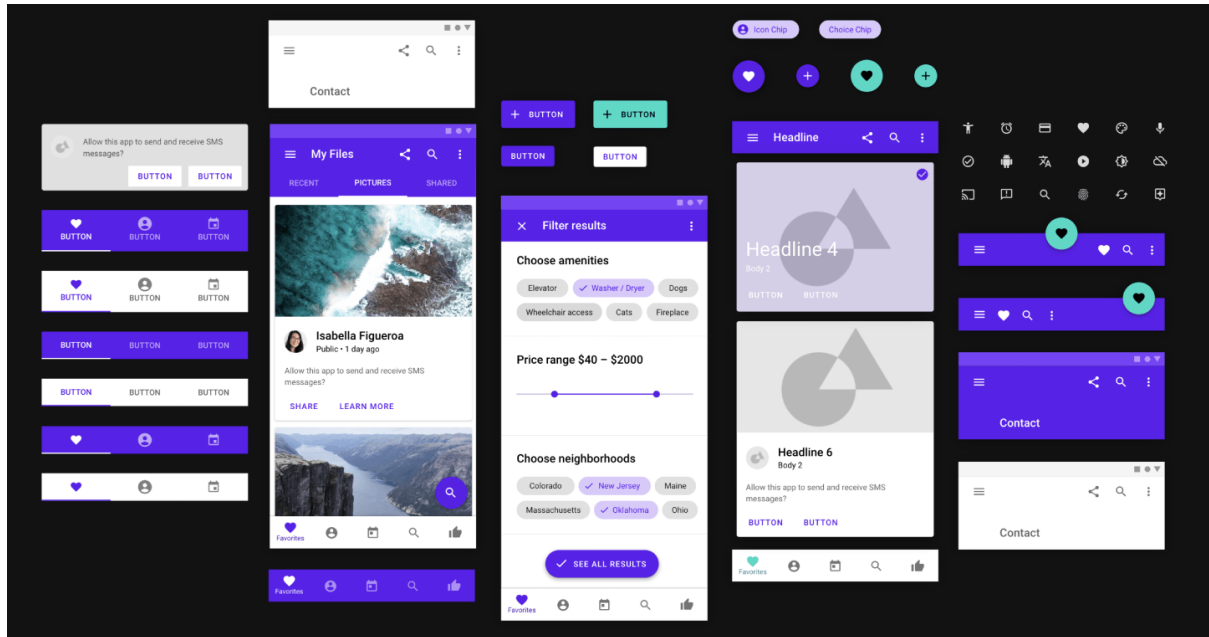
RecyclerView memiliki:

- Adapter

- ViewHolder
- LayoutManager

11.12 Material Design

Google memperkenalkan Material Design untuk menciptakan UI konsisten.



Komponen Material Design:

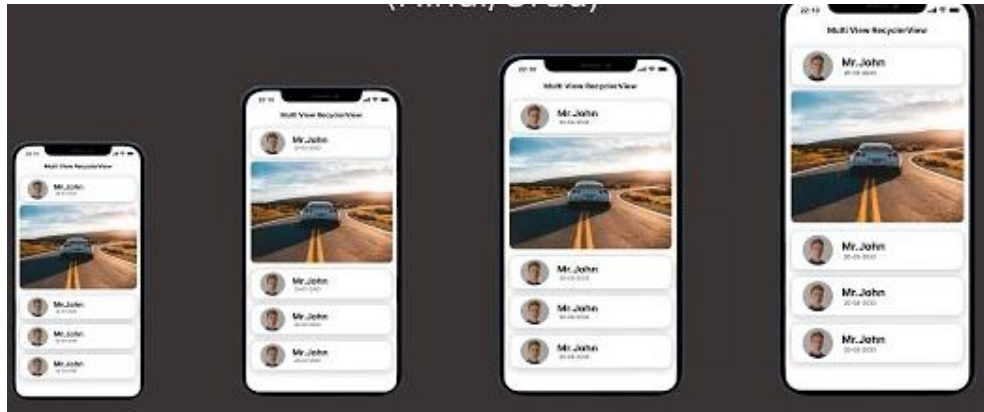
- Floating Action Button
- CardView
- Bottom Navigation
- Snackbar
- Navigation Drawer

11.13 Responsive UI

Android memiliki banyak variasi ukuran layar. Karena itu UI harus responsif. Pengembang harus mempertimbangkan:

- Smartphone
- Tablet

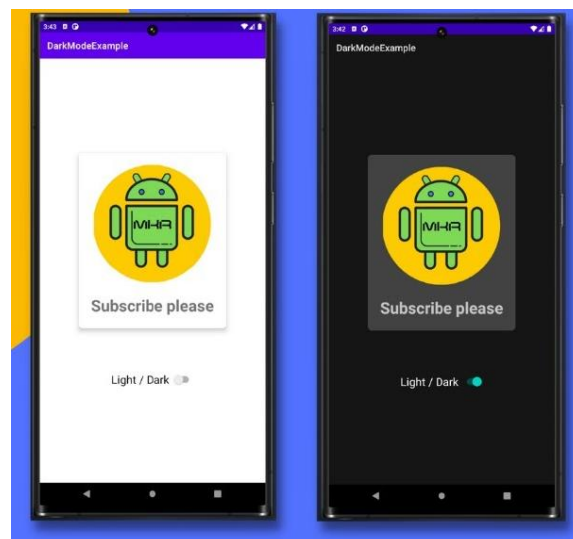
- Foldable device



11.14 Dark Mode

Android modern mendukung dark mode.

Dark mode membantu menghemat baterai dan meningkatkan kenyamanan.

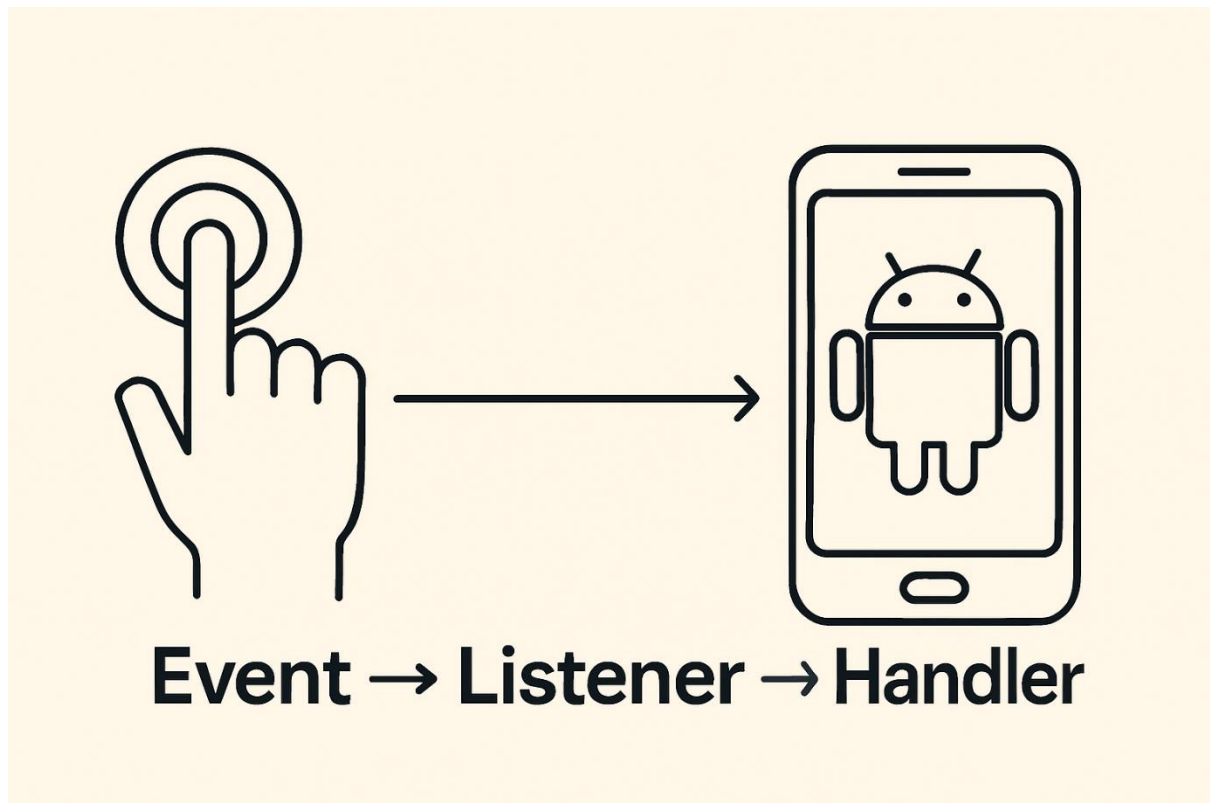


11.15 Kesimpulan Layout dan View

Layout dan View merupakan fondasi desain aplikasi Android.

Pemahaman yang baik terhadap UI sangat penting untuk menghasilkan aplikasi modern dan nyaman digunakan.

BAB XII EVENT-DRIVEN PROGRAMMING



12.1 Pengertian Event-Driven Programming

Event-driven programming merupakan paradigma pemrograman yang berorientasi pada event atau kejadian. Android sangat bergantung pada event-driven programming. Setiap interaksi pengguna seperti klik tombol, swipe, scroll, dan sentuhan layar merupakan event.

12.2 Event pada Android

Event merupakan aksi yang memicu proses tertentu.

Contoh event:

- Klik tombol
- Input keyboard
- Scroll layar
- Gesture
- Sensor berubah
- Lokasi berubah

12.3 Event Listener

Event listener digunakan mendeteksi event.

Contoh:

```
button.setOnClickListener {  
    Toast.makeText(this, "Klik", Toast.LENGTH_SHORT).show()  
}
```

Listener akan dipanggil ketika event terjadi.

12.4 Click Event

Click event merupakan event paling umum.

Digunakan pada:

- Button
- CardView
- RecyclerView item

12.5 Long Click Event

Long click terjadi ketika pengguna menekan komponen dalam waktu lama. Biasanya digunakan untuk menu tambahan.

12.6 Touch Event

Touch event digunakan untuk gesture. Android mendukung multitouch.

12.7 Callback Function

Android menggunakan callback pada banyak sistem.

Contoh callback lifecycle:

```
override fun onPause() {  
    super.onPause()  
}
```

Callback dipanggil otomatis oleh sistem.

12.8 Event Queue

Android menggunakan event queue untuk mengelola event.Event akan diproses satu per satu pada main thread.Jika event terlalu berat, aplikasi dapat mengalami lag.

12.9 UI Thread

UI Thread merupakan thread utama Android.Seluruh update UI harus dilakukan pada UI Thread.

Proses berat tidak boleh dijalankan pada UI Thread.

12.10 Asynchronous Programming

Untuk menghindari lag digunakan asynchronous programming. Android modern menggunakan:

- Coroutines
- Flow
- RxJava
- WorkManager

12.11 Event Handling pada RecyclerView

RecyclerView menggunakan event handling untuk interaksi item. Contoh:

- Klik item
- Swipe delete
- Drag and drop

12.12 Gesture Detection

Android mendukung gesture detection. Contoh:

- Swipe
- Pinch zoom

- Double tap
- Fling

12.13 Kesimpulan Event-Driven

Event-driven programming merupakan dasar interaksi aplikasi Android.

Pemahaman event sangat penting agar aplikasi responsif dan nyaman digunakan.

BAB XIII INTENT

13.1 Pengertian Intent

Intent merupakan mekanisme komunikasi antar komponen Android. Intent memungkinkan Activity, Service, dan Broadcast Receiver saling berinteraksi. Intent juga digunakan untuk navigasi aplikasi.

13.2 Jenis Intent

Intent terbagi menjadi dua jenis utama:

- Explicit Intent
- Implicit Intent

13.3 Explicit Intent

Explicit Intent digunakan untuk berpindah ke komponen tertentu. Contoh:

```
val intent = Intent(this, DetailActivity::class.java)
startActivity(intent)
```

Explicit Intent banyak digunakan pada navigasi internal aplikasi.

13.4 Implicit Intent

Implicit Intent digunakan untuk menjalankan aksi tertentu.

Contoh:

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = Uri.parse("https://google.com")
startActivity(intent)
```

Android akan memilih aplikasi yang sesuai.

13.5 Mengirim Data Antar Activity

Intent dapat digunakan untuk mengirim data. Contoh:

```
intent.putExtra("nama", "Ikmal")
```

Menerima data:

```
val nama = intent.getStringExtra("nama")
```

13.6 Intent Filter

Intent Filter digunakan menentukan jenis Intent yang dapat diterima komponen. Intent Filter dideklarasikan pada AndroidManifest.xml.

13.7 Pending Intent

Pending Intent memungkinkan aplikasi lain menjalankan Intent atas nama aplikasi tertentu. Pending Intent sering digunakan pada:

- Notification
- AlarmManager
- Widget

13.8 Intent dan Navigasi Modern

Android modern menggunakan Navigation Component. Namun konsep Intent tetap sangat penting.

Navigation Component membantu pengelolaan navigasi lebih aman.

13.9 Intent dan Keamanan

Intent dapat menjadi celah keamanan jika tidak dikelola dengan baik.

Pengembang harus:

- Membatasi exported component
- Validasi data Intent
- Menggunakan permission

13.10 Deep Link

Deep Link memungkinkan aplikasi dibuka melalui URL tertentu.

Deep Link penting untuk:

- Integrasi web
- Marketing
- Navigasi cepat

13.11 Share Intent

Android mendukung share Intent. Contoh:

```
val sendIntent = Intent().apply {  
    action = Intent.ACTION_SEND  
    putExtra(Intent.EXTRA_TEXT, "Hello")  
    type = "text/plain"  
}
```

13.12 Kesimpulan Intent

Intent merupakan komponen inti Android.

Tanpa Intent, komunikasi antar komponen tidak dapat berjalan.

Pemahaman Intent sangat penting untuk membangun aplikasi Android modern.

BAB VIV PENUTUP

Mobile programming merupakan bidang yang sangat penting dalam dunia teknologi informasi modern. Perkembangan perangkat mobile telah mengubah cara manusia berkomunikasi, bekerja, belajar, dan berinteraksi. Berbagai platform mobile seperti Android, iOS, Symbian, dan lainnya memiliki karakteristik serta keunggulan masing-masing. Android unggul dalam fleksibilitas dan pangsa pasar, sedangkan iOS unggul dalam stabilitas dan keamanan. Perkembangan framework modern seperti Flutter dan bahasa pemrograman Kotlin menunjukkan bahwa teknologi mobile terus mengalami inovasi. Mahasiswa Teknik Informatika diharapkan mampu memahami konsep mobile programming secara mendalam agar dapat berkontribusi dalam pengembangan teknologi masa depan. Dengan memahami sejarah, platform, framework, arsitektur, dan tantangan mobile programming, mahasiswa dapat memiliki dasar yang kuat untuk menjadi pengembang aplikasi mobile profesional.

DAFTAR PUSTAKA

1. Android Developers Documentation.
2. Flutter Official Documentation.
3. Apple Developer Documentation.
4. Sommerville, Ian. Software Engineering.
5. Pressman, Roger S. Software Engineering: A Practitioner's Approach.
6. Deitel, Paul. Android for Programmers.
7. Bill Phillips. Android Programming: The Big Nerd Ranch Guide.
8. Raywenderlich Tutorials.
9. Google Codelabs.
10. Oracle Java Documentation.